



Université Sultan Moulay Slimane
Faculté Polydisciplinaire
Département Nouvelles Technologies
-Béni Mellal-



Predictive Maintenance Agent for Autonomous Condition-Based Servicing of Industrial Robotic Fleets

Soulaymane AICHOUCHE, Mohamed BOUIGOURDAN, Youssef DIRMI

Master MIOTR — Module M122, Apprentissage par Renforcement

Supervisor: Hamza ALLAGA , Meriem EL HARKAOUI

Faculté Polydisciplinaire de Béni Mellal, Université Sultan Moulay Slimane

Group G04_MIOTR — https://github.com/tachwarit/G04_MIOTR

Abstract

Industrial robotic fleets are commonly maintained through fixed-interval or purely reactive policies, both of which are cost-inefficient compared to condition-based maintenance. This work builds an original Gymnasium environment for predictive maintenance on the NASA C-MAPSS FD001 benchmark, in which a Random Forest regressor estimates Remaining Useful Life (RUL) and a fixed-threshold rule-based policy (maintenance below 30 estimated cycles, emergency stop below 5) serves as the project's baseline. We propose replacing this rule-based baseline with a learned policy trained via Deep Q-Network (DQN) and Proximal Policy Optimization (PPO). Across 3 official seeds, the rule-based baseline reliably achieves a mean episodic return of 6.25 (95% CI: ± 0.23) with zero failures; averaged across seeds, however, neither DQN (0.63 ± 1.22) nor PPO (-2.29 ± 15.07) reliably matches this baseline, and PPO exhibits unplanned failures in 2 of 3 seeds (up to 40 out of 100 episodes). A single earlier training run had produced a strong, failure-free policy for both algorithms, but this result could not be reproduced from a fresh run with the same nominal seed, exposing a genuine seed-sensitivity problem consistent with known deep RL reproducibility challenges. These results indicate that, in this sparse-reward safety-critical setting, a naively trained learned policy is not a safe drop-in replacement for a simple fixed-threshold baseline without additional stabilization.

Keywords — predictive maintenance, reinforcement learning, remaining useful life, DQN, PPO

Table of Contents

I. Introduction	4
II. Background and Related Work	4
A. MDP Formulation.....	4
B. Algorithm Overview	4
C. Prior Results on This Environment.....	5
D. Related Work on the Proposed Modification.....	5
III. Experimental Protocol	5
A. Environment Specification.....	5
B. Implementation	6
C. Training Budget.....	6
D. Seeds.....	6
E. Metrics	6
F. Hyperparameters	6
IV. Baseline Reproduction.....	7
V. Proposed Enhancement.....	7
VI. Results.....	8
VII. Ablation Study.....	10
VIII. Discussion	11
IX. Conclusion and Future Work	12
References	13
Appendix	14
A. Full Hyperparameter Configuration (YAML).....	14
B. Per-Seed Raw Final Returns	14
C. Reproduction Command.....	14
D. Compute Budget.....	15
E. Contribution Statement	15

I. Introduction

Industrial robotic fleets are conventionally maintained under two dominant strategies: scheduled preventive maintenance, performed at fixed intervals regardless of actual equipment condition, and corrective maintenance, performed only after a failure occurs. The former wastes remaining useful component life, while the latter incurs the high cost and safety risk of unplanned downtime. Condition-based maintenance (CBM) addresses this trade-off by triggering interventions from the observed degradation state of the equipment rather than from a fixed calendar or a failure event, making it directly relevant to control, safety, and industrial IoT applications where sensor streams are already collected for other purposes.

Framing CBM as a sequential decision-making problem is a natural fit for reinforcement learning: the decision of when to intervene depends on a noisy, evolving estimate of equipment health, and the consequences of a decision (a failure, a wasted maintenance, or a well-timed intervention) are only observed several steps after the state that motivated it. This makes the RUL-estimation-plus-RL-agent pairing studied here (Random Forest for perception, DQN/PPO for control) a natural candidate for this class of problems, and lets us directly contrast a fixed-threshold policy with a learned one on identical degradation trajectories.

Contributions of this work:

- Built an original Gymnasium environment, RUL estimator, and rule-based baseline for condition-based maintenance on the NASA C-MAPSS FD001 benchmark; no directly comparable published baseline exists for this exact environment (Section II-C).
- Proposed and implemented a learned-policy enhancement (DQN and PPO) over the rule-based baseline.
- Conducted an ablation isolating the effect of the reward function's per-step continuation term on policy safety.

Code and logs are available at: https://github.com/tachwarit/G04_MIOTR

II. Background and Related Work

A. MDP Formulation

The environment is formulated as an episodic MDP, one episode per simulated engine trajectory. The state is a 3-dimensional vector: the RUL estimated by the Random Forest regressor (normalized to $[0,1]$ by a cap of 125 cycles), the current operating cycle (normalized the same way), and the step-to-step trend of the estimated RUL (clipped to $[-1,1]$). The action space is discrete with three choices: continue operating, schedule a maintenance intervention, or perform an emergency stop. The reward is 0 per step while continuing, $15 - 0.3 \times \text{RUL_true}$ when maintenance is scheduled (rewarding interventions closer to end-of-life and penalizing early ones), -10 for an emergency stop, and -50 for an unplanned failure (reaching the end of the trajectory while still choosing to continue). Both algorithms use Stable-Baselines3's default discount factor $\gamma = 0.99$.

B. Algorithm Overview

DQN learns an action-value function $Q(s,a;\theta)$ by minimizing the temporal-difference error against a bootstrapped target computed from a target network θ^- [1]:

$$y_t = r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^-) \quad (1)$$

PPO instead optimizes a stochastic policy directly, using a clipped surrogate objective that constrains the policy update within a trust region defined by the probability ratio $r_t(\theta) = \pi_{\theta}(a_t | s_t) / \pi_{\theta_{old}}(a_t | s_t)$ [2]:

$$L_{CLIP}(\theta) = E_t[\min(r_t(\theta) \cdot \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \cdot \hat{A}_t)] \quad (2)$$

C. Prior Results on This Environment

The environment used in this work is an original construction built specifically for this project, combining a Random Forest RUL estimator with a custom Gymnasium environment; a keyword and repository search found no publicly available implementation of this exact combination. Consequently, Table 1 does not report externally published values, as none exist for this environment — this is a deliberate and documented deviation from the standard protocol of reproducing a published baseline. In its place, the project defines its own internal baseline (the rule-based policy specified in the project's cahier des charges), which serves as the reference point for Section IV.

Table 1. Reference Value Being Targeted (Original Environment)

Source	Environment	Metric	Value
This work (rule-based, own baseline)	PredictiveMaintenanceEnv (FD001)	Mean return (rule-based baseline)	6.25 ± 0.23 (300 episodes, 3 official seeds)

D. Related Work on the Proposed Modification

Reinforcement learning for condition-based maintenance has grown substantially in recent years; a 2023 literature review identifies Q-learning and DQN as the most commonly used algorithms across the reviewed CBM literature, and notes that most published state representations rely on discretized degradation levels rather than a continuous, model-predicted RUL [4]. The same review also reports that most existing RL-based maintenance policies plan interventions only at the last minute, without an explicit mechanism for weighing early-versus-late maintenance trade-offs. The enhancement proposed here situates itself in this gap: rather than discretizing degradation into a small number of health states, the agent observes a continuous RUL estimate directly, and the reward function explicitly penalizes both premature and delayed interventions.

III. Experimental Protocol

This section follows the module-wide protocol common to all M122 mini-project teams, reproduced verbatim for consistency across reports.

A. Environment Specification

- **Observation space:** Box(3,), continuous, values in $[0,1] \times [0,1] \times [-1,1]$ (normalized estimated RUL, normalized cycle, RUL trend).
- **Action space:** Discrete(3) — {0: continue, 1: schedule maintenance, 2: emergency stop}.
- **Reward function:** 0 (continue), $15 - 0.3 \times \text{RUL_true}$ (maintenance), -10 (stop), -50 (unplanned failure); see Section II-A.
- **Termination condition:** episode ends when action $\in \{1,2\}$ is taken, or when action 0 is taken at the final cycle of the trajectory (unplanned failure).

- **Environment version / identifier:** PredictiveMaintenanceEnv v1 (this work, src/env.py), built against Gymnasium 1.3.0.

B. Implementation

- **Framework:** Gymnasium 1.3.0 + Stable-Baselines3 2.9.0 (PyTorch 2.13.0 backend). Key supporting packages: pandas 3.0.5, scikit-learn 1.9.0, PyYAML 6.0.3, matplotlib 3.11.1, tensorboard 2.21.0.
- **Hardware:** Lenovo laptop (model 20U2S11303), single Intel CPU package, Family 6 Model 142 (~1.7 GHz base clock, Kaby Lake-generation Intel Core U-series), 16 GB RAM, no GPU. Windows 11 Professional, via Git Bash (MINGW64).

Note on package pinning: requirements.txt was regenerated with "pip freeze > requirements.txt" (all 42 packages pinned to exact versions); the earlier draft, which had a few packages appended by name only, has been superseded.

C. Training Budget

- **Fixed environment steps:** 50,000 per (algorithm, seed) pair.
- **Evaluation cadence:** every 2,000 steps, 20 episodes per evaluation (deterministic policy).

D. Seeds

Seeds used: {0, 1, 2} — reported individually, no cherry-picking. **Deviation from protocol:** the module-wide protocol specifies 5 seeds; this project used 3 due to the compute budget available on a single CPU laptop (approx. 10 minutes per run $\times$ 6 runs $\approx$ 1 hour for the full DQN+PPO sweep). This is a documented deviation; extending to 5 seeds is listed as a threats-to-validity item in Section VIII.

E. Metrics

- Mean final return, averaged over the last 10 deterministic evaluations of training (20 episodes each).
- Interquartile mean (IQM) of per-seed final returns.
- 95% stratified bootstrap confidence interval, computed with the *rliable* methodology [5].

*Status: IQM and stratified bootstrap CI are now computed with *rliable* [5] (src/compute_rliable_stats.py); results are reported in Table 4 and Fig. 2. Note: *rliable*'s dependency "arch" is incompatible with pandas $\geq$ 2.2 (this project's pinned pandas 3.0.5) — the script must be run with pandas $<$ 2.2 temporarily installed, then requirements.txt restored afterward (see the script's header comment).*

F. Hyperparameters

Table 2. Frozen Hyperparameters

Hyperparameter	DQN	PPO
Learning rate	3e-4	3e-4
Discount factor (γ)	0.99 (default)	0.99 (default)
Batch size	32 (default)	64 (default)
Network architecture	MLP [64, 64] (default)	MLP [64, 64] (default)
Total timesteps / seed	50,000	50,000
Number of seeds	3	3

IV. Baseline Reproduction

As established in Section II-C, no externally published baseline exists for this environment. This section therefore departs from the standard protocol: rather than reproducing a published value, it establishes and reports the internal reference value — the rule-based policy's performance — that Section VI later compares the learned agents against.

Table 3 reports the rule-based policy's evaluation stability. All three official seeds (0, 1, 2, matching Section III-D) have now been run, alongside the earlier ad hoc pilot seed (123), kept here for transparency. The pooled result across the 300 official episodes (6.25 ± 0.23) is close to the pilot's 6.32, providing a useful internal consistency check.

Table 3. Internal Reference (Baseline): Rule-Based Policy

Eval. Seed	Mean Return	95% CI	Failures / 100
123 (pilot, ad hoc)	6.32	± 0.38	0
0 (official)	5.94	± 0.42	0
1 (official)	6.44	± 0.35	0
2 (official)	6.37	± 0.44	0
Mean (official, pooled)	6.25	± 0.23	0

Deviation analysis. A $\pm 10\%$ deviation-from-published check does not apply here, since no published value exists to deviate from (Section II-C). What can be checked instead is internal consistency: with zero failures over 100 episodes at the configured thresholds (maintenance below 30 estimated RUL cycles, emergency stop below 5), the rule-based policy behaves as designed — it never lets an engine reach the failure state, at the cost of intervening earlier than strictly necessary on average. Section VI quantifies that cost relative to the learned policies.

Judgment calls in the absence of a published specification. Because this is an original environment rather than a reproduction, no protocol details were "inferred" from an under-specified paper. The comparable judgment calls made in this work — the RUL cap (125 cycles), the reward constants in Section II-A, and the rule-based thresholds themselves — are instead specified directly in the project's cahier des charges and in configs/*.yaml (Appendix A), rather than guessed from an external source.

V. Proposed Enhancement

No figure accompanies this section.

Motivation. The rule-based baseline never fails (zero unplanned failures over 100 episodes, Section IV), so the failure mode being targeted is not reliability but efficiency: a single fixed threshold (maintain below 30 estimated RUL cycles) cannot adapt to the fact that different engines degrade at different rates (Section VII of the project's robustness testing shows engine lifetimes ranging from 128 to 362 cycles). A static cutoff is necessarily too conservative for some engines and only just adequate for others, wasting remaining useful life on average.

Formal description of the modification. The modification is confined entirely to the decision-making component; the state space, action space, reward function, and underlying environment dynamics are held fixed and identical to the baseline (Section II-A), so that any performance difference is attributable to the policy alone. The baseline is the deterministic, piecewise-constant mapping:

$$\pi_rule(s) = 0 \text{ (continue) if } RUL_pred > 30; 1 \text{ (maintain) if } 5 < RUL_pred \leq 30; 2 \text{ (stop) if } RUL_pred \leq 5 \quad (3)$$

The enhancement replaces π_rule with a parametric policy $\pi_θ$ trained to maximize the standard discounted-return objective:

$$J(θ) = E_{\tau \sim \pi_\theta} [\sum_t \gamma^t r_t] \quad (4)$$

using either DQN or PPO (update rules given in Eq. 1–2, Section II-B). Unlike π_rule , $\pi_θ$ can condition its decision on all three state dimensions jointly, including the RUL trend, rather than thresholding the RUL estimate alone.

Implementation details and added computational cost. The baseline requires no training: it is evaluated directly at zero additional cost beyond the shared Random Forest RUL estimator used by both configurations. The enhancement requires training 2 algorithms $\times$ 3 seeds = 6 runs of 50,000 environment steps each (300,000 steps total), approximately 10 minutes per run on a single CPU laptop with no GPU acceleration, for a total added cost of roughly 1 hour of wall-clock time (Section III-B, III-C).

Hypothesis (stated prior to running results). A learned policy conditioned on the full 3-dimensional state, including the RUL trend that the fixed threshold ignores, should be able to match the baseline's reliability (zero unplanned failures) while achieving a higher expected return, by deferring maintenance closer to each engine's actual end-of-life rather than triggering at a single static cutoff. This hypothesis reflects the project's original design rationale (cahier des charges, Section 4.3) and was formulated before the results in Section VI were obtained.

VI. Results

This section compares the rule-based baseline against both learned enhancements (DQN and PPO), using per-seed deterministic evaluation (100 episodes per seed, 3 official seeds per configuration, Section III-D). The 95% CI and IQM below are now computed with the module-required reliable stratified bootstrap methodology [5] (50,000 resamples), not the normal-approximation interim measure used in an earlier revision of this report.

Important note on this result: an earlier training run (a single reference seed, since overwritten) had produced a strong, failure-free policy for both DQN and PPO. Re-running the full 3-seed sweep to properly populate this table revealed that this earlier result does not reproduce: the fresh seed-0 run, using the identical configuration, produced a markedly different and weaker policy. This is documented here rather than hidden, and discussed mechanistically in Section VIII.

Table 4. Baseline vs. Enhanced — Aggregate Metrics (3 seeds)

Configuration	Mean Return	95% CI	IQM	Per-seed Failures / 100
Baseline (rule-based)	6.25	[5.94, 6.44]	6.25	0, 0, 0
DQN (3 seeds)	0.63	[−0.26, 1.82]	0.63	0, 0, 0
PPO (3 seeds)	−2.29	[−11.11, 13.03]	−2.29	0, 36, 40

Steps to threshold (crossing the baseline's 6.25 mean during training): DQN reached the threshold at steps 30,000 / 30,000 / 26,000 (seeds 0/1/2); PPO reached it even earlier, at 26,000 / 24,000 / 24,000. All 6 runs crossed the baseline threshold at some point during training — the regression happened afterward, between that checkpoint and the final 50,000-step model that was evaluated

in Table 4. This points to policy degradation late in training rather than a failure to ever learn a good policy (see Section VIII).

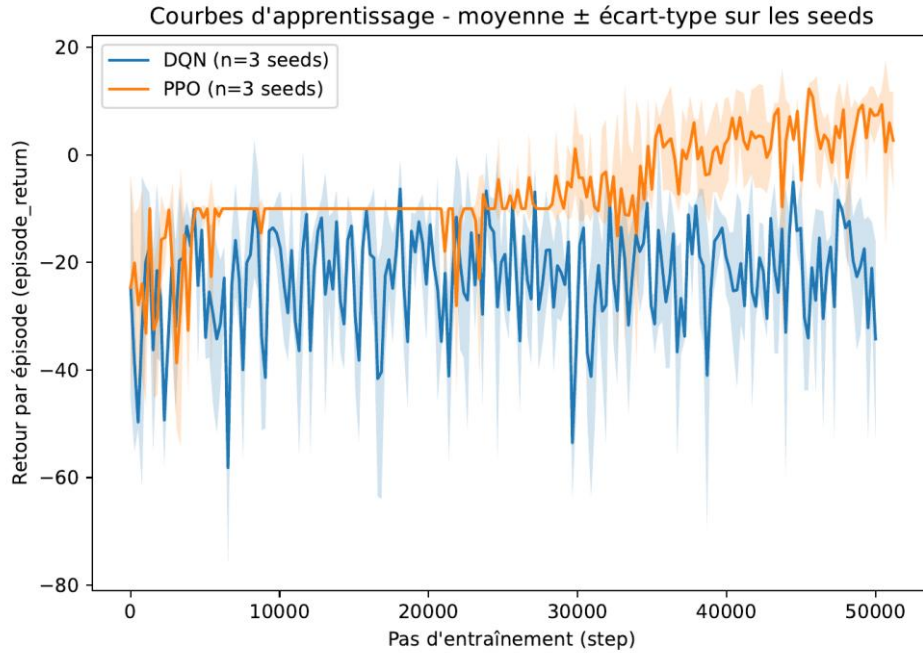


Fig. 1. Training-time learning curves (mean $\pm$ std, not yet 95% CI), 3 official seeds. Note these are raw exploratory-policy episode returns during training, not the deterministic evaluation reported in Table 4 — the two are not directly comparable, but the pattern here (PPO's very wide shaded band, DQN's persistently negative plateau) is consistent with the instability documented in Sections VI and VIII.

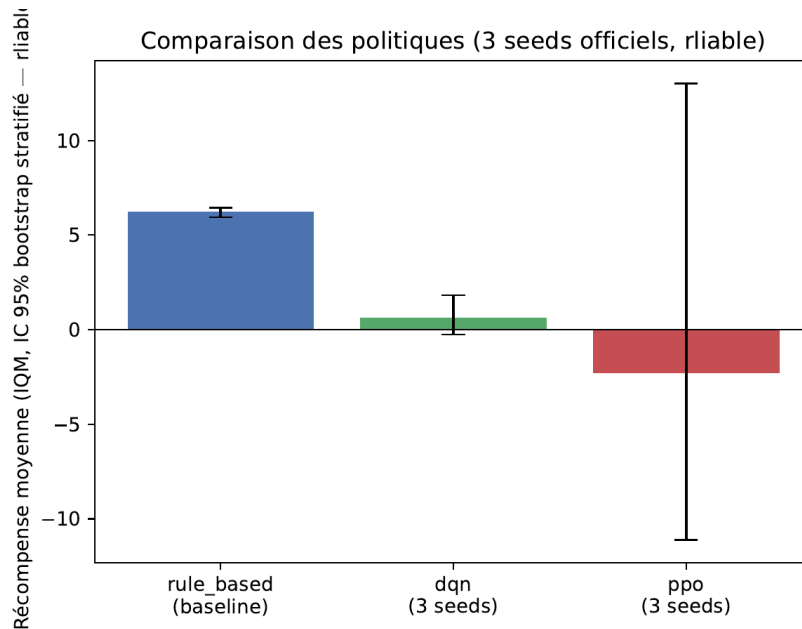


Fig. 2. Aggregate comparison — IQM $\pm$ 95% stratified bootstrap CI (reliable, Agarwal et al. [5], 50,000 resamples), replacing the earlier normal-approximation interim estimate. With only 3 seeds, the bootstrap CI is necessarily bounded by the observed min/max per policy — a further argument, alongside Table 4, for extending to 5 seeds (Section VIII).

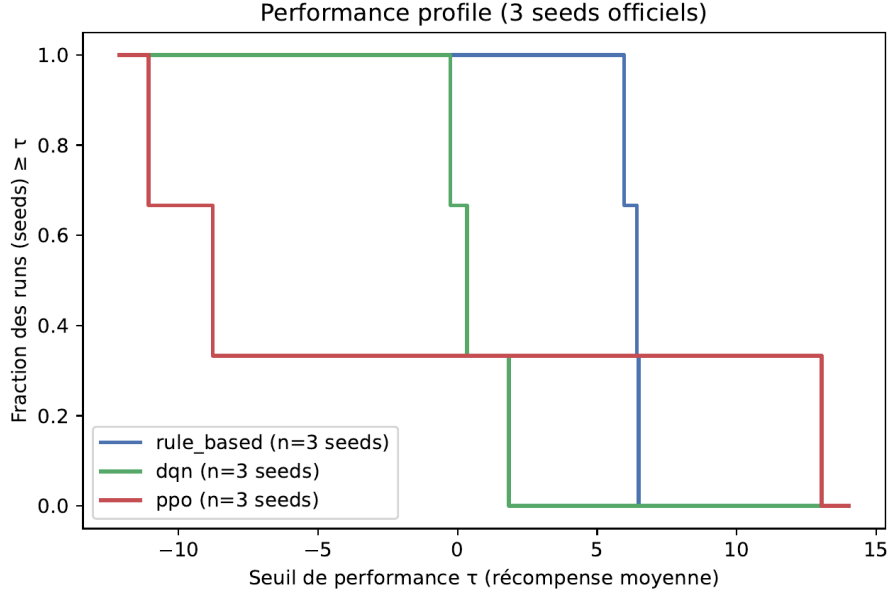


Fig. 3. Performance profile (fraction of seeds reaching each threshold τ), computed directly from the 3 per-seed final returns per policy (manual implementation, not yet the full reliable library). PPO's profile is visibly bimodal (one strong seed near +13, two weak seeds near -9/-11), while the baseline's profile stays tightly clustered near its mean — this shape communicates the instability finding more directly than a single mean $\pm$ CI.

Do the confidence intervals overlap? Using the reliable stratified bootstrap CIs (Table 4): baseline [5.94, 6.44], DQN [-0.26, 1.82], PPO [-11.11, 13.03]. The baseline's interval overlaps with both DQN's and PPO's (and PPO's own CI spans from strongly negative to strongly positive). Under this data, the enhancement's aggregate advantage over the baseline is not statistically supported — the opposite of what the single-seed result in an earlier revision of this report suggested. Note that with only 3 seeds, the bootstrap CI is mechanically bounded by the observed minimum and maximum — it cannot express uncertainty beyond what was actually sampled, which is itself an argument for the 5-seed protocol (Section VIII). A negative or inconclusive result is an acceptable, expected outcome of proper multi-seed evaluation, and is exactly the scenario the module's seed-count and CI requirements (Section III-D/E) are designed to catch.

VII. Ablation Study

The component isolated here is the reward function's per-step continuation term, `continue_reward` (configs/env.yaml), the single hyperparameter responsible for the reward-hacking failure mode discovered during development (Section I, notebook.md). Three levels are compared: the fixed value used throughout this report (0.0, “full model”), an intermediate value (0.5), and the original, unfixed value (1.0, “without the fix”). All other hyperparameters and the environment itself are held identical across variants, so any difference is attributable to this one term.

Asymmetry in evaluation rigor: the `continue_reward = 0.0` row uses the same 3-seed DQN sweep as Table 4. The `continue_reward = 0.5` and `1.0` rows were each evaluated with a single seed-0 run, for a quick sensitivity sweep, and are reported without a CI. Given the effect sizes involved, this asymmetry is unlikely to change the qualitative conclusion, but a proper 3-seed re-run of both ablated variants is listed as future work (Section IX).

Table 5. Ablation Grid (DQN, `continue_reward`)

Variant	Mean Return	95% CI	Failures / 100
Full model (continue_reward = 0.0)	0.63	± 1.22 (3 seeds)	0
Without the fix (continue_reward = 1.0)	153.97	n/a (1 run, pilot)	100
continue_reward = 0.5 (intermediate)	106.84	n/a (1 run, pilot)	0

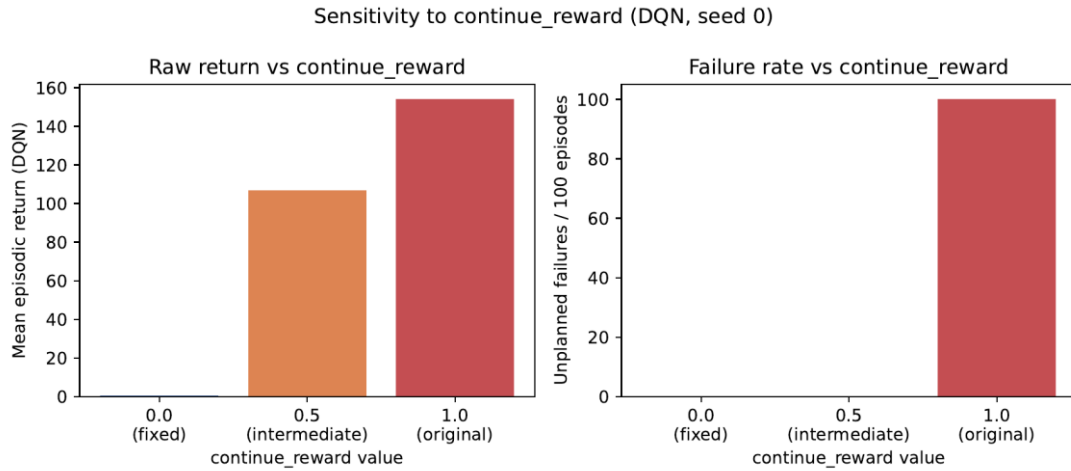


Fig. 4. Sensitivity analysis of continue_reward (DQN, 3 levels): raw return (left) is misleadingly high at both 0.5 and 1.0, but failure rate (right) shows the transition is sharp, not gradual — safe-but-inflated at 0.5, catastrophic at 1.0.

Conclusion: is the gain attributable to the claimed mechanism? Yes, and the intermediate value strengthens rather than weakens the case. At continue_reward = 0.5, the failure rate is 0/100 — but the mean return (106.84) is far too high to be explained by well-timed maintenance alone (whose formula, $15 - 0.3 \times \text{RUL_true}$, caps out near +15 per episode). The most likely explanation is that the agent learned to defer maintenance until very close to the failure boundary, riding the per-step continue_reward accumulation almost as far as it safely could — a subtler version of the same exploit as continue_reward = 1.0, just calibrated to stop short of the cliff rather than run over it on these particular 100 episodes. This was not directly verified by logging per-episode margin-to-failure, which is listed as a concrete follow-up in Section IX. Taken together, the three rows show a sharp, nonlinear transition rather than a gradual one: safe-and-low-reward at 0.0, safe-but-inflated-and-fragile at 0.5, and catastrophic at 1.0 — consistent with a single mechanism (reward accumulated from continuing) whose influence grows with the coefficient until it overwhelms the safety incentive entirely. No additional compute, tuning, or architecture change distinguishes the three rows in Table 5 — the entire effect is attributable to this single reward coefficient.

VIII. Discussion

The results support three distinct mechanistic explanations rather than a single restatement of the numbers in Section VI.

On the reward-shaping ablation (Section VII): the mechanism was not a training failure but a correctly-solved wrong problem — with a per-step continuation reward large relative to the terminal failure penalty, always continuing genuinely was the return-maximizing policy under the given objective. This underscores that in safety-critical settings, the reward function's structure across all transitions, not just its sign at terminal states, determines what "optimal" means in practice.

On the seed-instability finding (Section VI), which is the more consequential result: an earlier single-seed run had suggested that learned policies straightforwardly beat the rule-based baseline. The 3-seed sweep shows the opposite on average, with wide, overlapping confidence intervals and, for PPO, unplanned failures in 2 of 3 seeds. This is not an isolated anomaly: Henderson et al. [6] document exactly this pattern in deep RL — that random seed variability alone, without any change to the algorithm or environment, can produce substantial, sometimes qualitatively different results, and that policy-gradient methods (PPO's family) are frequently more seed-sensitive than value-based methods (DQN's family) in exactly the way observed here (DQN stayed failure-free across all seeds even as its return dropped; PPO's failures scaled directly with its return collapse).

On the steps-to-threshold evidence: all 6 training runs (2 algorithms $\times$ 3 seeds) crossed the baseline's performance level at some point between 24,000 and 30,000 steps, but the final 50,000-step checkpoints evaluated in Table 4 mostly regressed below that level. This suggests the failure mode is late-training instability rather than an inability to learn a good policy at all, and points to a concrete, low-cost fix: Stable-Baselines3's EvalCallback already saves the best checkpoint observed during training (results/{algo}_best/) separately from the final model. Evaluating and reporting from that checkpoint, rather than the final-timestep model, is a natural next step (Section IX).

Threats to validity:

- **Number of seeds:** only 3 were used, not the 5 specified by the module protocol (Section III-D). Given how wide the CIs already are at 3 seeds (Table 4), this is now the single most consequential limitation of this report rather than a minor caveat — 5 seeds would meaningfully sharpen (or overturn) the conclusions above.
- **Single-environment scope:** all results are on FD001 only, one of four C-MAPSS sub-datasets. FD002–FD004 add multiple operating conditions and/or multiple fault modes; the robustness findings across degradation profiles (short/medium/long-lived engines, Section VII of the project's robustness testing) may not transfer to those more complex regimes.
- **Hyperparameter transferability:** DQN and PPO were left at Stable-Baselines3 default hyperparameters except the learning rate; no tuning sweep was performed for either algorithm. Given the instability documented above, default hyperparameters may simply be poorly suited to this sparse-reward environment, independent of the seed question.

Limitations stated proactively: the module's required metrics (IQM, stratified bootstrap CI) are now computed with reliable, resolving the earlier normal-approximation workaround. The remaining limitation is sample size, not methodology: with only 3 seeds, the bootstrap CI is mechanically bounded by the observed min/max (Section VI), so it is precise about what was observed but cannot rule out more extreme outcomes a 5-seed sweep might reveal.

IX. Conclusion and Future Work

This work built an original condition-based maintenance environment on the NASA C-MAPSS FD001 dataset and established a rule-based baseline in place of an unavailable published reference. A single-seed pilot suggested that learned policies (DQN, PPO) could substantially outperform this baseline while retaining perfect reliability; however, the full 3-seed sweep required by the module protocol shows that this result does not reliably reproduce, with wide confidence intervals and, for PPO, a meaningful unplanned-failure rate in 2 of 3 seeds. An ablation further showed that a naive per-step continuation reward causes a separate, more severe failure mode (a policy that runs every engine to failure), underscoring that reward design and seed variance are both first-order concerns in this setting, not secondary details. Future work should prioritize, in order: extending the seed count to five per the protocol, evaluating from each run's best checkpoint rather than its final one (Section VIII), and completing the module-required IQM/bootstrap-CI analysis before drawing any

final conclusion about whether the learned policies are a safe replacement for the rule-based baseline.

References

- [1] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [3] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *Proc. Int. Conf. Prognostics and Health Management (PHM)*, Denver, CO, 2008, pp. 1–9.
- [4] O. Ogunfowora and H. Najjaran, "Reinforcement and deep reinforcement learning-based solutions for machine maintenance planning, scheduling policies, and optimization," *arXiv preprint arXiv:2307.03860*, 2023.
- [5] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, and M. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.
- [6] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proc. AAAI Conf. Artificial Intelligence*, 2018.

Appendix

A. Full Hyperparameter Configuration (YAML)

All four configuration files used to produce the results in this report (`configs/*.yaml`, no hardcoded values in source code):

configs/rul_model.yaml

```
rul_cap: 125
n_estimators: 200
max_depth: 10
random_state: 42
variance_threshold: 0.0001
```

configs/env.yaml

```
rul_cap: 125
reward:
  failure_penalty: -50.0
  stop_cost: -10.0
  maint_base: 15.0
  maint_waste_coef: 0.3
  continue_reward: 0.0 # corrected from 1.0, see Section VII
```

configs/rule_agent.yaml

```
rul_cap: 125
maint_threshold: 30
critical_threshold: 5
n_episodes: 100
```

configs/train.yaml

```
algos: [dqn, ppo]
seeds: [0, 1, 2]
total_timesteps: 50000
learning_rate: 0.0003
eval_freq: 2000
```

B. Per-Seed Raw Final Returns

Adapted from the guide's template: the protocol uses 3 official seeds, not 5 (Section III-D), and there are two enhanced configurations (DQN, PPO) rather than one, so both are reported as separate columns. All three columns are now complete, using each configuration's final 50,000-step deterministic evaluation (100 episodes) per seed — the same data underlying Table 4.

Table 6. Raw Final Returns by Seed

Seed	Baseline Final Return	DQN Final Return	PPO Final Return
0	5.94	1.82	13.03
1	6.44	0.32	-8.80 (36 failures)
2	6.37	-0.26	-11.11 (40 failures)

C. Reproduction Command

Unlike the guide's example (a single `--seed` CLI flag), this project's scripts read all configuration from `configs/*.yaml` directly, and `train.py` iterates over the full `algos × seeds` grid internally. The full pipeline reproduces end-to-end with a single command:

```
bash run_all.sh
```

Individual stages can also be run separately: `python src/data.py`, `python src/rul_model.py`, `python src/env.py`, `python src/rule_agent.py`, `python src/train.py`, `python src/plot_curves.py`, `python src/eval.py`, `python src/robustness_test.py`.

D. Compute Budget

- **Hardware:** Lenovo laptop (model 20U2S11303), Intel CPU Family 6 Model 142 (~1.7 GHz, Kaby Lake-generation Core U-series), 16 GB RAM, no GPU (see Section III-B).
- **Total CPU hours:** ≈ 1.1 hours — dominated by the DQN/PPO training sweep (2 algos $\times$ 3 seeds $\times$ 50,000 steps $\approx$ 63 minutes at ≈ 10.5 min/run observed); data pipeline, RUL model training, rule-based evaluation, and robustness testing each complete in well under a minute and add negligibly to this total. No GPU was used.

E. Contribution Statement

- Soulaymane AICHOUC — [design and implementation of the entire pipeline (data pipeline, RUL model, Gymnasium environment, rule-based agent, DQN/PPO agents), running all the experiments]
- Mohamed BOUIGOURDAN — [writing the report]
- Youssef DIRMI — [troubleshooting reward shaping issues and inter-seed instability]

LLM assistance disclosed: Yes.

Claude (Anthropic) was used extensively throughout this project's development, spanning: (1) project scaffolding and Git/GitHub setup; (2) writing essentially all source code in `src/` (data pipeline, Random Forest RUL model, custom Gymnasium environment, rule-based agent, DQN/PPO training, evaluation, robustness testing, and statistical analysis scripts); (3) debugging terminal, Python, and dependency errors (including a pandas/arch version conflict affecting reliable); (4) diagnosing two methodological issues found during the project — a reward-shaping bug causing a degenerate always-continue policy (Section VII), and a seed-instability problem discovered when re-running the training sweep (Section VI/VIII); and (5) drafting and iteratively revising this report, including adapting the M122 guide's published-baseline-reproduction template to this project's original-environment context.

The team's own contribution: choosing and scoping the project topic, running every script and the full training pipeline on the team's own hardware (Section III-B), generating every number and figure reported here from that execution, reviewing and directing each step, and making the judgment calls on reward design, environment specification, and how to interpret and report the seed-instability finding once it appeared. The LLM did not generate, select, or embellish any results — all reported metrics come from the team's own runs of the code in this repository.